

CLI Agent JSON Schema Extraction Plan v0.1.1-hygiene

Machine-readable schema extraction, canonical object registry, validation phases, and implementation handoff for the C-Governed CLI Agent Mesh package

Status	Draft schema-extraction plan v0.1.1-hygiene synchronized
Date	2026-05-17
Package	C-Governed CLI Agent Mesh
Layer	c = a + b / Agent Governance / Schema Extraction / Machine Validation / Conformance / Witness
Document class	schema extraction plan / implementation bridge / machine-facing package-control artifact
Assertion class	C-A10 package-control artifact; C-A7 where canonicalization, hashing, signing, or schema-verification claims are made
Primary boundary	prose examples and YAML sketches are not executable schema authority until extracted, normalized, validated, versioned, and linked to semantic validation rules.

Contents

0. Executive definition	3
1. Purpose.....	3
2. Non-goals.....	4
3. Corpus bridge set	4
3.1 Explicit bridge: $c = a + b$	4
3.2 Quiet bridge I: information theory.....	5
3.3 Quiet bridge II: engineering tolerance	5
3.4 Earth paragraph.....	5
4. Source availability status	5
5. Schema policy.....	6
5.1 Recommended schema draft.....	6
5.2 File extension.....	6
5.3 Encoding	6
5.4 Schema ID convention	6
5.5 Object version convention.....	6
6. Canonical schema layout.....	7
7. Minimum viable P0 schema set.....	7
8. Shared definitions.....	9
9. Structural vs semantic validation.....	10
9.1 Structural validation	10
9.2 Semantic validation.....	11
10. Semantic validator modules	11
11. Red-line const constraints	12
12. SCHEMA_INDEX.json target shape	12
13. Extraction batches.....	13
13.1 Batch P0-A — gate and package-control base	13
13.2 Batch P0-B — agent admission and task execution base.....	13
13.3 Batch P0-C — review, memory, evidence, and interruption.....	13
13.4 Batch P0-D — release and conformance base.....	13
14. Example validation plan	14
15. Conformance fixture binding.....	14
16. Local checker binding	15
17. Codex extraction task contract.....	15
18. Review checklist for extracted schemas.....	16
19. Open issues.....	16
20. Closing rule	17

0. Executive definition

CLI Agent JSON Schema Extraction Plan defines how the object definitions embedded across the C-Governed CLI Agent Mesh Markdown documents should be extracted into machine-readable JSON Schema files.

v0.1.1 update: this plan is now synchronized with the expanded Schema Object Registry and Semantic Validator Rules. SCDP, CTM, and Package Index sources are available, not missing.

Compact formula:

```
Markdown defines the doctrine.  
The object registry names the objects.  
Schemas define the shape.  
Semantic validators define the boundary.  
Conformance tests prove behavior.
```

1. Purpose

The v0.1 package contains structured object sketches across many profiles:

```
readiness records  
AB/gate records  
registry entries  
task contracts  
capability profiles  
permission grants  
handshakes  
sandbox profiles  
witness events  
raw evidence sidecars  
memory gate records  
rollback/freeze records  
quorum and review records  
defensive emulation records  
incident records  
cloud data records  
release surface records  
local checker records  
conformance test records  
fixture records  
schema registry records
```

Most objects currently exist as Markdown-embedded YAML examples or prose-defined shapes.

That is enough for architectural review.

It is not enough for implementation.

This plan creates the bridge from:

```
protocol prose
-> schema object registry
-> JSON Schema files
-> SCHEMA_INDEX.json
-> example validation
-> semantic validation
-> conformance fixtures
-> local checker
-> implementation readiness
```

2. Non-goals

This plan does not:

1. replace the Markdown protocol documents;
2. define all cryptographic signing rules;
3. create a full reference implementation;
4. certify runtime safety;
5. prove legal compliance;
6. validate provider-specific data handling;
7. authorize any offensive or red-line behavior;
8. make schema validity equivalent to system safety;
9. claim conformance.

A JSON object may be structurally valid and still unsafe.

Therefore:

```
JSON Schema validation is necessary.
It is not sufficient.
```

3. Corpus bridge set

3.1 Explicit bridge: $c = a + b$

Schemas belong to b : procedures, validators, object shapes, logs, witnesses, memory records, conformance artifacts, and implementation tooling.

Schemas do not become c .

Schemas do not decide authority.

Schemas help prevent worker agents from smuggling authority through malformed or ambiguous objects.

3.2 Quiet bridge I: information theory

A schema reduces ambiguity. But a schema cannot decide whether an action is lawful, proportionate, or safe in context. That requires semantic validation and review.

3.3 Quiet bridge II: engineering tolerance

A part can match the drawing and still fail under load if the material or use case is wrong. JSON Schema is the drawing, not the load test.

3.4 Earth paragraph

A work order saying “fix the electrical panel” is not enough. A real work order has fields: panel ID, circuit, breaker, lockout state, responsible worker, inspection, and sign-off. But filling all fields does not mean the repair is safe; someone still checks the actual circuit. JSON Schema gives the form. Conformance tests check the circuit.

4. Source availability status

Source profile	Status	Notes
CLI_A-gent_Secrets_and_Cloud_Data_Policy_v0_1.md	source_available	Required for SCDP schemas.
CLI_A-gent_Conformance_Test_Matrix_v0_1.md	source_available	Required for CTM schemas.
CLI_A-gent_Pack-age_In-dex_and_Reading_Order_v0_1.md	source_available	Recreated in synchronized set.
CLI_A-gent_Schema_Object_Registry_v0_1.md	source_available	Controls object inventory.
CLI_A-gent_Semantic_Validator_Rules_v0_1.md	source_available	Controls semantic validation map.
CLI_A-gent_Local_Checker_Profile_v0_1.md	source_available	Controls local checker object family and run result.

No SCDP/CTM object should be marked `blocked_source_missing` after this synchronization.

5. Schema policy

5.1 Recommended schema draft

Use:

```
JSON Schema Draft 2020-12
```

5.2 File extension

Use:

```
.schema.json
```

5.3 Encoding

Use:

```
UTF-8
```

5.4 Schema ID convention

Use stable package-local `$id` until repository URL is finalized.

Temporary v0.1 pattern:

```
urn:cgams:schema:<object-name>:0.1
```

Example:

```
"$id": "urn:cgams:schema:cli-agent-task-contract:0.1"
```

5.5 Object version convention

Object instances should include:

```
"schema_version": "<object-name>-0.1"
```

6. Canonical schema layout

Recommended root:

```
schemas/
  README.md
  SCHEMA_INDEX.json
  common/
    cgam-common-defs-0.1.schema.json
  readiness-gate/
  ab-gate-semantics/
  registry/
  task-contract/
  permission-capability/
  handshake/
  sandbox-worktree/
  witness/
  raw-evidence-sidecar/
  memory-gate/
  rollback-freeze/
  quorum-review/
  executor-reviewer-separation/
  defensive-emulation/
  incident-response/
  secrets-cloud-data/
  local-checker/
  release-public-surface/
  public-redaction/
  conformance/
  fixtures/
  schema-registry/
```

7. Minimum viable P0 schema set

The minimum implementation-ready extraction batch must include:

Object	Family	Target schema file
CLI_AGENT_READINESS_RECORD	readiness-gate	readiness-gate/cli-agent-readiness-record-0.1.schema.json
CLI_AGENT_RELEASE_COMPLETION_RECORD	readiness-gate	readiness-gate/cli-agent-release-completion-record-0.1.schema.json
CLI_AGENT_IMPLEMENTATION_HANDOFF_RECORD	readiness-gate	readiness-gate/cli-agent-implementation-handoff-record-0.1.schema.json
CLI_AGENT_CLAIM_CONTROL_RECORD	readiness-gate	readiness-gate/cli-agent-claim-control-record-0.1.schema.json
CLI_AGENT_AB_MODE_DECLARATION	ab-gate-semantics	ab-gate-semantics/cli-agent-ab-mode-declaration-0.1.schema.json
CLI_AGENT_GATE_SEMANTICS_RECORD	ab-gate-semantics	ab-gate-semantics/cli-agent-gate-semantics-record-0.1.schema.json

Object	Family	Target schema file
CLI_AGENT_GATE_EVALUATION_RECORD	ab-gate-semantics	ab-gate-semantics/cli-agent-gate-evaluation-record-0.1.schema.json
CLI_AGENT_SCHEMA_OBJECT_REGISTRY	schema-registry	schema-registry/cli-agent-schema-object-registry-0.1.schema.json
CLI_AGENT_SCHEMA_OBJECT_ENTRY	schema-registry	schema-registry/cli-agent-schema-object-entry-0.1.schema.json
CLI_AGENT_REGISTRY	registry	registry/cli-agent-registry-0.1.schema.json
CLI_AGENT_REGISTRY_ENTRY	registry	registry/cli-agent-registry-entry-0.1.schema.json
CLI_AGENT_HANDSHAKE	handshake	handshake/cli-agent-handshake-0.1.schema.json
CLI_AGENT_TASK_CONTRACT	task-contract	task-contract/cli-agent-task-contract-0.1.schema.json
CLI_AGENT_PERMISSION_GRANT	permission-capability	permission-capability/cli-agent-permission-grant-0.1.schema.json
CLI_AGENT_SANDBOX_PROFILE	sandbox-worktree	sandbox-worktree/cli-agent-sandbox-profile-0.1.schema.json
CLI_AGENT_WITNESS_EVENT	witness	witness/cli-agent-witness-event-0.1.schema.json
CLI_AGENT_RAW_EVIDENCE_SIDECAR	raw-evidence-sidecar	raw-evidence-sidecar/cli-agent-raw-evidence-sidecar-0.1.schema.json
CLI_AGENT_MEMORY_GATE_RECORD	memory-gate	memory-gate/cli-agent-memory-gate-record-0.1.schema.json
CLI_AGENT_FREEZE_RECORD	rollback-freeze	rollback-freeze/cli-agent-freeze-record-0.1.schema.json
CLI_AGENT_REVIEW_RECORD	quorum-review	quorum-review/cli-agent-review-record-0.1.schema.json
CLI_AGENT_DATA_CLASSIFICATION_RECORD	secrets-cloud-data	secrets-cloud-data/cli-agent-data-classification-record-0.1.schema.json
CLI_AGENT_LOCAL_CHECKER_PROFILE	local-checker	local-checker/cli-agent-local-checker-profile-0.1.schema.json
CLI_AGENT_CHECKER_RUN	local-checker	local-checker/cli-agent-checker-run-0.1.schema.json
CLI_AGENT_CHECKER_RESULT	local-checker	local-checker/cli-agent-checker-result-0.1.schema.json
CLI_AGENT_RELEASE_SURFACE_RECORD	release-public-surface	release-public-surface/cli-agent-release-surface-record-0.1.schema.json

Object	Family	Target schema file
CLI_AGENT_PUBLIC_PACKAGE_MANIFEST	release-public-surface	release-public-surface/cli-agent-public-package-manifest-0.1.schema.json
CLI_AGENT_RELEASE_GATE_RECORD	release-public-surface	release-public-surface/cli-agent-release-gate-record-0.1.schema.json
CLI_AGENT_PUBLIC_RESTRICTED_SPLIT_RECORD	release-public-surface	release-public-surface/cli-agent-public-restricted-split-record-0.1.schema.json
CLI_AGENT_CONFORMANCE_RESULT	conformance	conformance/cli-agent-conformance-result-0.1.schema.json

This P0 set is intentionally larger than the original v0.1 extraction plan because v0.1.1 added load-bearing readiness, AB/gate, registry, raw-evidence, local-checker, release-surface, schema-registry, and semantic-validator layers.

8. Shared definitions

Create:

```
schemas/common/cgam-common-defs-0.1.schema.json
```

Required shared definitions:

```
schema_version
object_id
created_at
updated_at
governing_entity_id
human_anchor_ref
agent_ref
task_contract_ref
permission_grant_ref
sandbox_ref
witness_ref
evidence_sidecar_ref
artifact_hash_ref
release_surface_ref
decision
risk_class
sensitivity_class
retention_class
gate_status
checker_status
claim_status
```

Common decision enum:

```
allowed
denied
held
blocked
frozen
quarantined
revoked
expired
completed
failed
accepted
rejected
needs_review
red_line_fail
```

Implementation claim enum:

```
draft_complete
release_completion_in_progress
release_complete
implementation_readiness_in_progress
implementation_ready
implementation_executed
conformance_supported
public_release_ready
```

No object may assert a stronger claim than readiness/release/conformance gates support.

9. Structural vs semantic validation

9.1 Structural validation

JSON Schema can enforce:

- required fields;
- allowed enum values;
- object shapes;
- array types;
- boolean const constraints;
- string formats;
- conditional requirements;
- absence of additional fields.

9.2 Semantic validation

JSON Schema cannot fully enforce:

- whether a path is actually denied;
- whether a command is destructive in context;
- whether data is truly public or private;
- whether cloud routing actually occurred;
- whether provider retention policy is current;
- whether a reviewer is genuinely independent;
- whether a quorum is same-source;
- whether evidence was preserved before repair;
- whether an immunity update is too broad;
- whether legal review is required;
- whether output contains hidden prompt injection;
- whether a live target is truly owned/authorized;
- whether AB_MODE=B was accompanied by all required gates.

Therefore, extracted schemas must bind to
CLI_Agent_Semantic_Validator_Rules_v0_1.md.

10. Semantic validator modules

Recommended modules:

```
validate_readiness_claim_semantics
validate_ab_gate_semantics
validate_registry_semantics
validate_task_contract_semantics
validate_permission_semantics
validate_handshake_semantics
validate_sandbox_semantics
validate_witness_semantics
validate_raw_evidence_sidecar_semantics
validate_memory_gate_semantics
validate_rollback_freeze_semantics
validate_quorum_review_semantics
validate_executor_reviewer_separation_semantics
validate_defensive_emulation_semantics
validate_incident_response_semantics
validate_secrets_cloud_semantics
validate_release_surface_semantics
validate_public_redaction_semantics
validate_local_checker_semantics
validate_conformance_semantics
validate_fixture_semantics
```

11. Red-line const constraints

Where applicable, schemas should include const-false constraints for prohibited behaviors.

Recommended shared properties:

```
{
  "direct_memory_write_allowed": { "const": false },
  "self_approval_allowed": { "const": false },
  "witness_tampering_allowed": { "const": false },
  "secret_prompting_allowed": { "const": false },
  "live_external_exploitation_allowed": { "const": false },
  "hack_back_allowed": { "const": false },
  "autonomous_retaliation_allowed": { "const": false },
  "malware_behavior_allowed": { "const": false },
  "credential_theft_allowed": { "const": false },
  "covert_persistence_allowed": { "const": false }
}
```

Omission must not mean allow. Semantic validators must treat omission as deny unless the object family has no relevance to that risk.

12. SCHEMA_INDEX.json target shape

The extracted schema package should include:

```
{
  "schema_version": "cli-agent-schema-index-0.1",
  "package": "C-Governed CLI Agent Mesh",
  "package_version": "0.1",
  "generated_at": "YYYY-MM-DDTHH:MM:SSZ",
  "registry_source": "CLI_Agent_Schema_Object_Registry_v0_1.md",
  "schemas": [
    {
      "object": "CLI_AGENT_TASK_CONTRACT",
      "schema_version": "cli-agent-task-contract-0.1",
      "schema_file": "schemas/task-contract/cli-agent-task-contract-0.1.schema.json",
      "priority": "P0",
      "source_profile": "CLI_Agent_Task_Contract_Schema_v0_1.md",
      "status": "schema_extracted",
      "semantic_validator": "validate_task_contract_semantics",
      "restricted": false
    }
  ]
}
```

Required fields for every `schemas[]` entry:

```
object
schema_version
schema_file
priority
source_profile
status
```

```
semantic_validator  
restricted
```

13. Extraction batches

13.1 Batch P0-A — gate and package-control base

```
CLI_AGENT_READINESS_RECORD  
CLI_AGENT_RELEASE_COMPLETION_RECORD  
CLI_AGENT_IMPLEMENTATION_HANDOFF_RECORD  
CLI_AGENT_CLAIM_CONTROL_RECORD  
CLI_AGENT_AB_MODE_DECLARATION  
CLI_AGENT_GATE_SEMANTICS_RECORD  
CLI_AGENT_GATE_EVALUATION_RECORD  
CLI_AGENT_SCHEMA_OBJECT_REGISTRY  
CLI_AGENT_SCHEMA_OBJECT_ENTRY
```

13.2 Batch P0-B — agent admission and task execution base

```
CLI_AGENT_REGISTRY  
CLI_AGENT_REGISTRY_ENTRY  
CLI_AGENT_HANDSHAKE  
CLI_AGENT_TASK_CONTRACT  
CLI_AGENT_PERMISSION_GRANT  
CLI_AGENT_SANDBOX_PROFILE  
CLI_AGENT_WITNESS_EVENT
```

13.3 Batch P0-C — review, memory, evidence, and interruption

```
CLI_AGENT_RAW_EVIDENCE_SIDEAR  
CLI_AGENT_MEMORY_GATE_RECORD  
CLI_AGENT_FREEZE_RECORD  
CLI_AGENT_REVIEW_RECORD  
CLI_AGENT_LOCAL_CHECKER_PROFILE  
CLI_AGENT_CHECKER_RUN  
CLI_AGENT_CHECKER_RESULT
```

13.4 Batch P0-D — release and conformance base

```
CLI_AGENT_RELEASE_SURFACE_RECORD  
CLI_AGENT_PUBLIC_PACKAGE_MANIFEST  
CLI_AGENT_RELEASE_GATE_RECORD  
CLI_AGENT_PUBLIC_RESTRICTED_SPLIT_RECORD  
CLI_AGENT_DATA_CLASSIFICATION_RECORD  
CLI_AGENT_CONFORMANCE_RESULT
```

14. Example validation plan

For every Markdown file, extract YAML/JSON examples into:

```
examples/<profile>/<object>/<example-name>.yaml
```

Each example should validate structurally and semantically.

Example categories:

```
valid/  
invalid/  
red-line/  
edge-case/
```

Invalid examples should include expected result:

```
deny  
hold  
quarantine  
revoke  
red_line_fail
```

15. Conformance fixture binding

Every conformance fixture should specify:

```
fixture_metadata:  
  fixture_id: string  
  schema_ref: string  
  semantic_validator: string  
  expected_structural_result: pass | fail  
  expected_semantic_result: pass | fail | red_line  
  sensitivity: public_synthetic | restricted_synthetic | internal_redacted  
  profiles_tested:  
    - CATC  
    - CAPM
```

A fixture must not contain real secrets, live targets, exploit recipes, deployable malware behavior, credential theft logic, evasion procedures, or retaliation workflows.

16. Local checker binding

The local checker must verify:

1. every P0 schema file exists;
2. every P0 schema uses correct `schema_version` const;
3. every schema file path matches SOR;
4. every object in `SCHEMA_INDEX.json` exists in SOR;
5. every P0 object has semantic validator entry or explicit blocker;
6. no restricted object is exposed as public-safe by default;
7. duplicate source profiles are resolved before extraction;
8. no schema permits red-line behavior through missing booleans;
9. no release/implementation/conformance claim exceeds gate records.

If any P0 object is missing after extraction, checker result must be:

```
FAIL or BLOCKED
```

not `PASS_WITH_WARNINGS`.

17. Codex extraction task contract

A safe Codex task for schema extraction should look like:

```
task_title: Extract P0 JSON schemas for CGAM v0.1.1 hygiene set
scope:
  allowed_paths:
    - docs/cli-agent-mesh/
    - schemas/
    - examples/
    - validator/
  denied_paths:
    - secrets/
    - private_memory/
    - legal/
    - incident_evidence/
    - production/
    - .git/
  allowed_actions:
    - read canonical Markdown source profiles
    - create schema files under schemas/
    - create SCHEMA_INDEX.json
    - create validation report
    - create discrepancy report
  prohibited_actions:
    - modify doctrine meaning
    - infer missing source profiles
    - access secrets
    - fetch external material
    - publish
```

```

- tag release
- push without review
- claim conformance
output_required:
- created schema list
- validation report
- unresolved questions
- diff
- rollback plan
review_required: true
memory: off
auto_ingest: false

```

18. Review checklist for extracted schemas

For each schema:

- ☐ Filename matches SOR.
- ☐ `$schema` is Draft 2020-12.
- ☐ `$id` follows package convention.
- ☐ `schema_version` const is correct.
- ☐ `additionalProperties` is set intentionally.
- ☐ Required fields match Markdown profile.
- ☐ Enum values match glossary / SOR shared defs.
- ☐ Red-line booleans use `const: false` where appropriate.
- ☐ Human gate requirements are represented structurally where possible.
- ☐ Semantic rules are listed where JSON Schema cannot enforce them.
- ☐ Valid examples pass.
- ☐ Invalid examples fail.
- ☐ Red-line examples fail safely.
- ☐ Schema is referenced in `SCHEMA_INDEX.json`.

19. Open issues

ID	Issue	Required action	Status
JSEP-01-001	Final repo URL unknown.	Use URN <code>\$id</code> until repository placement is finalized.	OPEN

ID	Issue	Required action	Status
JSEP-0I-002	Retention class families are not fully harmonized.	Resolve or keep profile-local explicitly before shared defs.	DEFERRED
JSEP-0I-003	Some examples contain prose command abstractions.	Normalize examples during extraction.	OPEN
JSEP-0I-004	Signature/canonicalization not fully specified.	Defer to future signature/canonicalization profile.	DEFERRED
JSEP-0I-005	Provider-specific schema fields may change after provider profiles.	Keep provider-specific extensions modular.	DEFERRED
JSEP-0I-006	Semantic validators are specified but not implemented.	Create validation module plan/stubs.	OPEN
JSEP-0I-007	Fixture pack exists but is not bound to runner.	Bind after P0/P1 schemas.	OPEN
JSEP-0I-008	Cross-object validation needs object registry.	SOR exists; create SCHEMA_INDEX.json.	PARTIALLY_RESOLVED

20. Closing rule

Schema extraction is not paperwork.

It is the point where protocol language becomes machine boundary.

Final rule:

If the object shape is vague,
the worker will improvise.

If the worker improvises authority,
the mesh is not governed.